

DEPARTAMENTO DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico

Implementación de TAD: Sistema de Fidelización AirMiles

November 8, 2026 — Algoritmos y Estructuras de Datos

Grupo 1

Integrante	LU	Correo electrónico
Garcia Ocio, Santiago	1210/25	santiagogarciaocio@gmail.com
Seda, Manuel	631/18	manu.seda@hotmail.com
García Peruchini, Federico	638/25	fede.gp2018@gmail.com
Pascazzi, Martin	711/25	martinpascazzi@gmail.com

Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires
Ciudad Universitaria - (Pabellón I/Planta Baja)
Intendente Güiraldes 2610 - C1428EGA
Ciudad Autónoma de Buenos Aires - Rep. Argentina
Tel/Fax: (++54 +11) 4576-3300
http://www.exactas.uba.ar

1 Diseño

El TAD AirMilles fue implementado con el siguiente diseño:

- MaxHeap de Usuarios.

Esta estructura es vital para cumplir las complejidades requeridas para la operación rankingMillas. Como trade-off de agregar el Heap surgió un problema: Vincular a los usuarios con sus posiciones en el Heap. Recordemos que el Heap es O(1) para obtener el máximo, pero para cualquier otro elemento es O(U).

Un primer intento para solucionar este problema consistió en agregar a cada usuario un atributo con su posición en el Heap, logrando así ir a su posición en O(1).

Sin embargo al momento de hacer siftUp o siftDown, operaciones necesarias para mantener el invariante de la estructura, cada vez que un usuario acumula o canjea millas, habría que modificar el índice de todos los usuarios afectados por el shift.

Por lo tanto necesitábamos poder acceder y modificar, desde la clase Heap, el índice de los usuarios en O(1) para cualquier usuario. Para lograr esto, decidimos que el heap no contenga simplemente a los usuarios, sino a nodos cuyas componentes son un elemento, que en nuestro caso son los usuarios, y un ID para identificarlos.

Además, tenemos un array de posiciones, cuyos índices representan el ID del usuario, por ejemplo, posiciones[1] nos dice en qué posición del heap está el usuario con ID 1 (para esto definimos una clase Vector ya que ArrayList no te deja agregar en la posición 10 si no tenés nada en la 0 a la 9, y queremos que los IDs puedan ingresar en cualquier orden). De esta manera podemos intercambiar las posiciones en el heap de 2 usuarios en tiempo constante, así que una vez que tenemos el ID del usuario al que queremos aumentar o disminuir sus millas, acomodarlo en el heap será O(log U) en el peor caso.

- ABB que guarda los handles de los Usuarios.

El ABB está conectado con el heap via handle, y ordenado según el ID de los usuarios. Como tenemos un ABB y no un AVL encontrar a un usuario por ID va a ser O(U) en el peor caso, pero O(log U) en el caso promedio.

- Trie de Promociones.

Implementado en forma de arreglo. En particular tenemos nodos de longitud 256. Cada posición del nodo representa un caracter. Cuando se ingresa una nueva promoción cada caracter crea nuevos nodos iterativamente. Al finalizar el nombre de la promoción, el atributo 'Elem' pasa a contener un elemento de la clase promoción.

Para poder eliminar promociones de forma tal que se eliminen todos los nodos que no llevan a ninguna promoción, utilizamos el atributo palabrasEnRama. Éste cuenta la cantidad de palabras que pasan por él. Una vez que dicho atributo nos marca 1, nos indica que a partir de ese momento la única promoción en ese camino del Trie es la que queremos borrar.

- MasAcumulador.
  - El usuario que tiene más millas totales.
- MasCanjeador.
  - El usuario que gastó más millas.

Promocion y Usuario son clases con sus correspondientes atributos y métodos.

2 Complejidades

Veamos que este diseño nos permite cumplir con todas las complejidades requeridas para cada operación.

- crearSistema en O(1)
  - Inicializar ABB O(1)
  - Inicializar Heap O(1)
  - Inicializar Trie O(1)
  - agregarPromocion('NoPromo') O(7) = O(1)
  - Costo total: O(1)
- registrarUsuario / registrarUsuarioConReferidor en O(log U) caso promedio y O(U) peor caso
  - Encolamos Usuario en el Heap O(log U)
  - Agregamos el Handle del usuario al ABB; caso promedio O(log U), peor caso O(U)
  - Sumarle millas al usuario referidor es O(1) ya que es lo que cuesta es encontrarlo
  - En el peor caso vamos vamos a tener que actualizar la posicion en el Heap del usuario referidor del piso al tope. Eso tiene costo de O(log U)
  - Caso promedio: O(log U) +O(log U) +O(1) + O(log U) = O(log U)
  - Peor caso: O(log U) +O(U) +O(1) + O(log U) = O(U)
- acumularMillas en O(log U + |L|) caso promedio y O(U + |L|) peor caso
  - Encontrar la promocion en el Trie: O(|L|)
  - Obtener la informacion de la promocion, restar del tope si es necesario O(1)
  - Eliminar promoción en caso de ser necesario O(|L|) [ver eliminarPromocion]
  - Encontrar al usuario en el ABB : caso promedio O(log U), peor caso O(U)
  - Aumentar los atributos correspondientes al usuario O(1)
  - Reajustar la posición del Usuario en el Heap, de ser requerido, O(log U)
  - Caso promedio: O(|L|) + O(1) + O(|L|) + O(log U) + O(1) + O(log U) = O(log U + |L|)
  - Peor caso: O(|L|) + O(1) + O(|L|) + O(U) + O(1) + O(log U) = O(U + |L|)
- canjearMillas O(log U) caso promedio y O(U) peor caso
  - Encontrar al usuario en el ABB: caso promedio O(log U), peor caso O(U)
  - Realizar la operación canjearMillas de la clase Usuario O(1)
  - Caso promedio: O(log U) + O(1) = O(log U)
  - Peor caso: O(U) + O(1) = O(U)
- transferirMillas en O(log U) caso promedio y O(U) peor caso
  - Encontrar al usuario transferidor en el ABB: caso promedio O(log U), peor caso O(U)
  - Encontrar al usuario receptor en el ABB: caso promedio O(log U), peor caso O(U)
  - Realizar el balance correspondiente del saldo de Millas de transferidor O(1)
  - Alterar el Heap para reflejar las nuevas millas del transferidor O(log U)
  - Actualizar, si fuera necesario, MasCanjeador O(1)

  - Realizar el balance correspondiente del saldo de Millas del receptor O(1)
  - Alterar el Heap para reflejar las nuevas millas del receptor O(log U) en el peor caso
  - Caso promedio: O(log U) + O(log U) + O(1) + O(log U) + O(1) + O(1) + O(log U) = O(log U)
  - Peor caso: O(U) + O(U) + O(1) + O(log U) + O(1) + O(1) + O(log U) = O(U)

- consultarSaldo / consultarCategoria en O(log U) caso promedio y O(U) peor caso
  - Encontrar al usuario en el ABB : caso promedio O(log U), peor caso O(U)
  - Consultar su saldo/categoría O(1)
  - Caso promedio: O(log U) + O(1) = O(log U)
  - Peor caso: O(U) + O(1) = O(U)

- iniciarPromocion / iniciarPromocionConTope en O(|L|)
  - Insertar elemento en Trie O(|L|)
  - En el peor caso por cada letra nueva hay que crear una nueva 'rama' del Trie. Esto tiene costo fijo dado por la longitud del arreglo O(256)
  - Crear Promocion con o sin tope O(1)
  - Costo total: O(|L|) + O(1) = O(|L|)

- finalizarPromocion en O(|L|)
  - Buscar la promoción en el Trie O(|L|)
  - Eliminar la promoción si solamente se tiene que eliminar la promocion O(1), si se tienen que eliminar ramas O(|L|)
  - Costo total en el peor caso: O(|L|) + O(|L|) = O(|L|)

- elMasAcumulador / elMasCanjeador en O(1):
  - Pedirle información a MasAcumulador O(1)
  - Pedirle información a MasCanjeador O(1)
  - Costo total: O(1)

- rankingMillas en O(N log U)
  - Tomar N veces el máximo del Heap O(N log U)
  - Copiar los usuarios a un array O(N)
  - Volver a ingresar N veces los usuarios al Heap O(N log U)
  - Costo total: O(N log U) + O(N) + O(N log U) = O(N log U)
  - (Observación: si hay menos de N usuarios en el sistema, se reemplaza N por la cantidad total de usuarios, por lo que sigue valiendo la cota O(N log U))

- paresTransferidoresExclusivos en O(U)
  - Recorrer el ABB elemento por elemento O(U)
  - Chequear para cada usuario si está en parTransferidorExclusivo O(1)
  - Costo total: O(U)